

Vibecoding With ALCF Inference

Agentic coding workflows with open models on ALCF hardware

Ethan Wong

September 28, 2026



Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.



Argonne Leadership
Computing Facility



U.S. DEPARTMENT
of ENERGY

Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.

Introduction



Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.



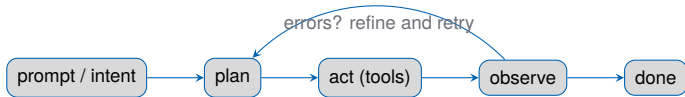
Argonne Leadership
Computing Facility

What is Vibecoding?

- Term coined by Andrej Karpathy (Feb 2025):
“There’s a new kind of coding I call ‘vibe coding’, where you fully give in to the vibes, embrace exponentials and forget that the code even exists.”
- **Working definition:** you describe what you want in plain language, and a coding agent writes, runs, and debugs the code. You steer by reviewing the results instead of typing every line.
- You still do the hard part: deciding what to build and judging whether it is correct.
- For domain scientists: the barrier shifts from syntax to specification and verification, skills you already have.

From Chatbots to Coding Agents

- **2021–22:** IDE autocomplete brings Copilot suggestions into your editor, still line-by-line
- **2022:** chat assistants. You copy/paste snippets from the chat window, run everything, and fix every error
- **2024–25:** terminal agents that read/write files, run shell commands, search the repo, observe errors, and self-correct



The agent is now the executor. You become the architect and reviewer, like advising a tireless student.

The ALCF Inference Service



Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.



Argonne Leadership
Computing Facility

ALCF Inference Endpoints at a Glance

- **30+ open-weight models** on ALCF hardware, behind one OpenAI-compatible API
- Authenticate with your **ANL/ALCF credentials** via Globus
- Web UI: <https://inference.alcf.anl.gov>, or hit the API directly
- Billions of LLM tokens already served

Cluster	Hardware	Base path	Wire APIs
Sophia	NVIDIA A100	/resource_server/sophia/vllm/v1	chat, responses ¹ , messages ¹ , completions, embeddings
Metis	SambaNova SN40L	/resource_server/metis/api/v1	chat, responses ¹
Minerva	NVIDIA B200	/resource_server/minerva/api/v1	chat, responses, messages, completions, embeddings

¹Streaming for these APIs is disabled on Sophia and Metis; use Chat Completions or Minerva.

Setting Up Your Harness



Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.



Argonne Leadership
Computing Facility

Anatomy of a Coding Agent

- A **harness** is the terminal-native app wrapping the model with a tool belt, a loop, and guardrails:
 - **Tools:** read/edit/write files, run shell commands, search the codebase, (sometimes) the web
 - **Loop:** model proposes action → tool executes → output observed → repeat until done
 - **Guardrails:** approval prompts, sandboxing, git
- Popular harnesses. All support external providers, so they can point at ALCF Inference:
 - **OpenCode** (open source, today's default) and **Pi**, recommended in the ALCF docs
 - **Claude Code**, featured in the session right after this one
 - **OpenAI Codex**, and community shims/proxies for anything else (alc-f-proxy, llm-rosetta)

Picking a Model for Agents

- Agents talk to models through **tool calling**: the harness advertises tools (read file, run command, ...), the model replies with structured calls
- Capability flags in the docs: **B**atch, **R**easoning, **T**ool calling, always **H**ot
- Agents need models with **T**; **R** improves planning; **H** avoids cold starts

Model	Cluster	Flags	Why
openai/gpt-oss-120b	Sophia	BRTH	solid default, strong tool use
openai/gpt-oss-20b	Sophia	BRTH	faster, lighter tasks
google/gemma-4-31B-it	Sophia	RTH	strong reasoning
meta-llama/Llama-4-Scout-17B-16E-Instruct	Sophia	BT	long context

Full model list: `uvx alcf-ai ls-endpoints`. On Sophia, non-hot models may take 10–15 min to cold-start; check the Web UI or the `/jobs` endpoint.

Install the Agent

- One-liners install to ~/.local/bin; reload your shell afterwards

```
# OpenCode (today's default)
```

```
curl -fsSL https://opencode.ai/install | bash
```

```
# Claude Code (next session!)
```

```
curl -fsSL https://claude.ai/install.sh | bash
```

```
# reload your shell afterwards (zsh: ~/.zshrc; or open a new terminal)
```

```
source ~/.bashrc
```

- Also available via brew, apt, etc.
- Same ideas apply to Pi, Codex, and friends; see the #agents section of the ALCF Inference docs

One-Command Configuration: `alcf-ai`

- `alcf-ai` logs you in, fetches your token, and writes the provider config into your agent's settings

```
# install uv if you don't have it
curl -LsSf https://astral.sh/uv/install.sh | sh

# authenticate + configure (interactive Globus login on first use)
uvx alcf-ai agent configure opencode
```

- Supports opencode, pi, codex, claude
- **Idempotent:** re-run it any time to embed a fresh token
- Need the raw token for a script?
`export ALCF_AI_TOKEN="$(uvx alcf-ai auth get-access-token)"`

What Just Got Configured?

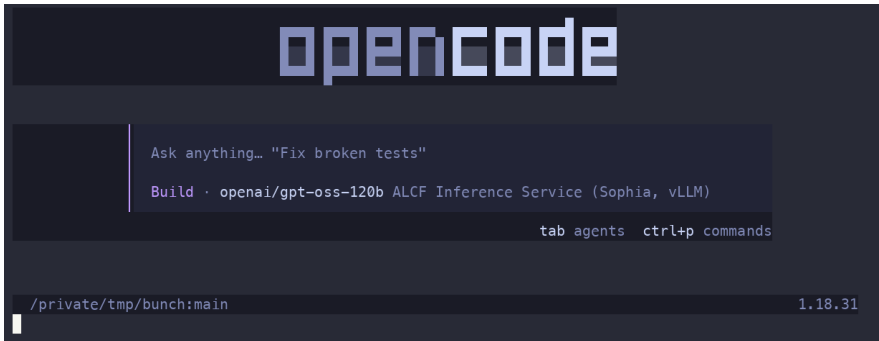
- OpenCode now knows about an ALCF provider (~/.config/opencode/opencode.jsonc):

```
{
  "provider": {
    "alcf-inference-service-sophia": {
      "npm": "@ai-sdk/openai-compatible",
      "options": {
        "baseUrl": "https://inference-api.alcf.anl.gov/resource_server/sophia/vllm/v1",
        "apiKey": "{env:ALCF_AI_TOKEN}"
      },
      "models": {
        "openai/gpt-oss-120b": { "name": "openai/gpt-oss-120b" }
      }
    }
  }
}
```

- Claude Code is similar: ANTHROPIC_BASE_URL + ANTHROPIC_AUTH_TOKEN in ~/.claude/settings.json (Minerva); full snippet in the #agents docs

Your First Session

- Run opencode inside your project; `/init` writes an `AGENTS.md` that it reuses on later turns
- The prompt box shows the active agent and model; `ctrl+p` lists every command



Watching the Agent Work

- Your prompt, the model's reasoning, and every **tool call** land in one transcript
- Here Glob found the file and Read opened it, so you can audit exactly what it saw

```
In two sentences: what does particles.py compute, and in what units?
```

```
Thought: 4ms
```

```
We need to answer question: In two sentences: what does particles.py compute, and in what units? Need to examine repository. Use glob to find particles.py.
```

```
* Glob "**/particles.py" (1 match)
```

```
Thought: 2ms
```

```
We need to read the file particles.py to see its content.
```

Plan First, Then Build: Tab

- Tab switches between the **Build** and **Plan** agents; the prompt box shows which is active
- **Plan** cannot edit files: it only reads and proposes, so a bad plan costs nothing

2. Implement median_kinetic_energy

```
def median_kinetic_energy(energies):  
    """Median kinetic energy, MeV, for energies given as totals."""  
    ke = [e - MASS_MEV for e in energies]    # convert to kinetic  
    return median(ke)                        # median from the statistics module
```

- Mirrors the pattern used in `mean_kinetic_energy` and `spread`: first strip the rest-mass energy, then compute the statistic.

3. Wire the function into the script's main block

- Inside `if __name__ == "__main__":` after loading the data, add:

```
med = median_kinetic_energy(e)  
print(f"median KE : {med:.3f} MeV")
```

- Keep the existing prints for particle count, mean KE, and spread.

4. Update documentation/comments (optional but low-cost)

Review the Diff

- Every edit arrives as a **diff** with line numbers: red is removed, green is added
- Review the diff, not the summary. One increment per prompt, then run the code

```
← Edit particles.py

29     return median(ke)
30
31
32 if __name__ == "__main__":
33 -     median_ke = median_kinetic_energy(e)
34 -     print(f"median KE : {median_ke:.3f} MeV")
33     e = load_energies("run_2041.dat")
34     print(f"particles : {len(e)}")
35     print(f"mean KE   : {mean_kinetic_energy(e):.3f} MeV")
36     print(f"spread   : {spread(e):.3f} MeV")
37 +     median_ke = median_kinetic_energy(e)
```

Undo and Restore

- `/undo` reverts the last change and hands your original prompt back so you can sharpen it
- `/redo` (or `ctrl+x r`) puts it back; `git` stays your real undo button

```
The above steps introduce the median calculation with minimal changes, maintain the existing API style, and keep all energy values consistently expressed in MeV.
```

```
Plan · openai/gpt-oss-120b · 4.9s
```

```
1 message reverted  
ctrl+x r or /redo to restore
```

```
Go ahead and implement the plan.
```

```
Build · openai/gpt-oss-120b ALCF Inference Service (Sophia, vLLM)
```

```
/private/tmp/bunch
```

```
10.3K (16%) ctrl+p commands
```

Vibecoding Effectively



Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.



Argonne Leadership
Computing Facility

Trick 1: Vibe in Small Loops

- **Plan before you build:** ask the agent to read the code and propose a plan first; review it, then let it code
- **One feature per prompt:** small steps, run the code after every change, commit after every success
- **Don't fight a confused agent:** if it spirals, restate the goal with clearer context, or start a fresh session instead of piling on corrections
- Git is your undo button; a clean history of small commits makes every agent edit cheap to review or revert

Manage the agent like a talented student: small, well-specified tasks beat vague epics.

Trick 2: Ground the Agent (Context is Everything)

- Keep an **instructions file** (AGENTS.md, CLAUDE.md, ...) at the repo root: project purpose, build/run/test commands, conventions, and no-go zones ('never touch src/legacy')
- **Point, don't paste:** tell the agent which files matter and let it read/search the rest; pasted code goes stale
- **Fresh session per task:** long transcripts rot the model's working memory (the context window) and the agent's judgment
- Encode your **domain knowledge** explicitly: units, file formats, expected values, edge cases. The model cannot guess your physics.

Trick 3: Make Success Verifiable

- Agents shine when correctness is checkable: tests, linters, type checkers, a run script that prints a known result
- **Failing-test-first**: “write a test that captures the requirement, then make it pass”. Great for scientific code
- Paste error messages back verbatim; the agent reads its own tracebacks far better than your paraphrase
- For science: golden outputs, sanity-limit checks, property tests. Verify numerics against results you trust.

Trick 4: Stay in Control

- **Review the diff, not the vibes:** you own what ships (papers, repos, production)
- Keep **humans on approval** for destructive or far-reaching commands; sandbox the agent when you can
- **Never paste tokens or credentials** into a prompt; keep secrets out of transcripts
- Models **hallucinate**: invented APIs, imaginary flags, plausible-but-wrong physics. Verify imports, citations, and numerics before they matter.
- Know when not to vibe: unverifiable correctness-critical code, security-sensitive paths

Crash Course



Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.



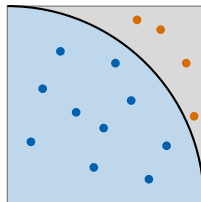
Argonne Leadership
Computing Facility

Your Mission: Monte Carlo π , With a Copilot

- Throw N random darts at the unit square; the fraction p landing in the quarter circle estimates its area:

$$\hat{\pi} = 4p, \quad \sigma_{\hat{\pi}} = 4\sqrt{p(1-p)/N}$$

- Small enough to finish in one session, yet it has everything real science code has: numerics, an error bar, a known answer, and a figure
- Same loop every step: **prompt** $\rightarrow$ **code** $\rightarrow$ **python pi_mc.py** $\rightarrow$ **look at the plot** $\rightarrow$ **refine**
- You are the architect; the agent is the executor; π is the referee



$$\hat{\pi} = 4 \times \text{inside} / \text{all}$$

Get Ready to Vibe

- **Checklist:** `setup.sh` has already installed `uv`, `OpenCode`, and the Python environment, logged you in, and set up `OpenCode` for ALCF

```
cd Service_Enabled_Science
./setup.sh                # safe to re-run; refreshes your token
source .venv/bin/activate  # numpy + matplotlib now on PATH

mkdir -p ~/pi-demo && cd ~/pi-demo
git init                  # your undo button
opencode                  # pick openai/gpt-oss-120b (Sophia)
```

- Use a new, empty directory so the agent can't find the reference `solution.py`
- The full prompt and the follow-ups are in `04_Coding_with_Inference_Service/PROMPT.md`

Anatomy of a Good Prompt

Write a single Python script, `pi_mc.py`, that estimates π with Monte Carlo sampling and makes a publication-quality figure of the result.

Environment:

- Use the `python` already on PATH (numpy + matplotlib). Do not pip install anything.
- Headless: savefig only, never `plt.show()`.

The math:

- N uniform points, `np.random.default_rng(seed), vectorized;  $\pi\_hat = 4p$ ;`
`stderr = 4*sqrt(p(1-p)/N).`
- Print the estimate $\pm$ stderr; fail if it is more than 3 stderr from `math.pi`.

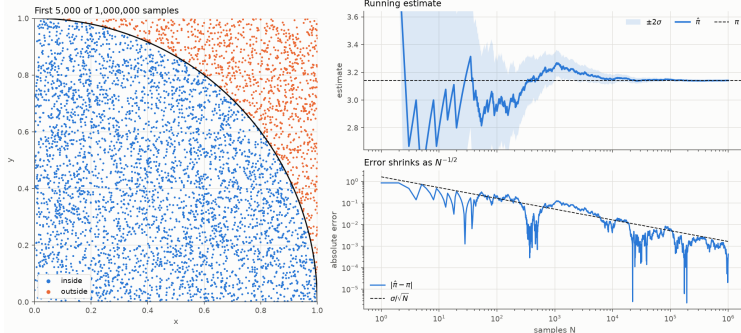
The figure: (1) dartboard scatter, (2) running estimate with a ± 2 sigma band on a log x-axis, (3) log-log error vs. a $\sigma/\sqrt{N}$ line.

Then run it twice, show me the output, and fix any errors before you report back.

- **Name the deliverable:** one file, one output
- **Pin the environment:** open models guess the environment wrong unless you tell them what's installed
- **Spell out the science:** formulas, units, and expected values; the model can't guess your physics
- **Describe the figure panel by panel:** "make it pretty" gives a generic plot
- **Build in a check:** the 3σ assert and "run it" let the agent fix its own mistakes

What Good Looks Like

Monte Carlo $\pi = 3.14117 \pm 0.00164$ (true $\pi = 3.14159$)



10^6 samples: $\hat{\pi} = 3.14117 \pm 0.00164$, which is 0.26σ from π . The error follows the $N^{-1/2}$ line, so each additional digit of π takes $100\times$ more samples.

Check the Agent's Work, Then Push Further

Verify

- Run it yourself: 10^6 samples should give an estimate within ~ 0.003 of π
- Open the PNG: dots split at the arc, the estimate settles onto π , the error slope matches $N^{-1/2}$
- Read the diff: no Python loop over samples, no `plt.show()`, no `pip install`
- Commit once it works

Then, one prompt at a time

- Run K independent seeds: is their spread consistent with the standard error?
- Time $N = 10^5 \dots 10^8$ and report how runtime scales
- Add a `pytest` check against π for several seeds
- Swap in a Sobol sequence (`scipy.stats.qmc`): how much faster does the error fall?

Troubleshooting

- **Access tokens expire after 48 h:** re-run `uvx alcf-ai agent configure <agent>`; re-authenticate at Globus roughly every 30 days
- **OpenAI Codex ≥ 0.95** speaks the Responses API only: pin 0.94 or lower on Sophia/Metis, or point Codex at Minerva
- **Responses and Messages streaming is disabled** on non-Direct-API clusters (Sophia, Metis); use Chat Completions or Minerva
- **Metis tool-calling is flaky** (SambaNova sanitization); prefer Sophia for agent workloads
- **Cold starts on Sophia:** non-hot models can take 10–15 min to load, so prefer **H** models (check the Web UI or `/jobs`)

References

- ALCF Inference Endpoints docs (incl. #agents):
<https://docs.alcf.anl.gov/services/inference-endpoints>
- ALCF Inference Endpoints repo & examples:
<https://github.com/argonne-lcf/inference-endpoints>
- alcf-ai on PyPI: <https://pypi.org/project/alcf-ai/>
- Harness homepages: OpenCode <https://opencode.ai/>; Pi <https://pi.dev/>; Claude Code <https://code.claude.com/docs/en/quickstart>; Codex <https://learn.chatgpt.com/docs/codex/cli>
- Karpathy, A. (Feb 2025): “vibe coding” post on X
- Tanikanti et al., *FIRST: Federated Inference Resource Scheduling Toolkit for Scientific AI Model Access*, SC Workshops '25. <https://doi.org/10.1145/3731599.3767346>
- Mailing list: inference-service-notify (
<https://lists.alcf.anl.gov/mailman/listinfo/inference-service-notify>)